

lab05

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(leaflet)
library(scales)
```

Anexando pacote: 'scales'

O seguinte objeto é mascarado por 'package:purrr':

discard

O seguinte objeto é mascarado por 'package:readr':

col_factor

A função `analisa_aeronave` recebe o `tail_number` da aeronave, o caminho do `flights.csv.zip` e o caminho do `airports.csv` (coordenadas dos aeroportos). Ela lê o arquivo de voos em pedaços, filtra só os voos da aeronave, calcula a velocidade média de cada trecho, junta as coordenadas

de origem/destino e monta um mapa leaflet do trajeto ao longo do ano. Retorna uma lista com \$tabela e \$mapa

```
analisa_aeronave = function(tail_number, arquivo, aeroportos = file.path("dados", "airports.csv")) {  
  
  # le o flights.csv.zip em pedacos, mantendo so os voos da aeronave  
  filtra_voos_da_aeronave = function(bloco, pos) {  
    bloco %>% filter(TAIL_NUMBER == tail_number)  
  }  
  
  voos_aeronave = read_csv_chunked(  
    file = arquivo,  
    callback = DataFrameCallback$new(filtra_voos_da_aeronave),  
    chunk_size = 50000,  
    col_types = cols(.default = "c") # tudo como texto, converte depois  
  )  
  
  # ajusta os tipos das colunas e monta a data-hora de partida pra poder ordenar  
  voos_aeronave = voos_aeronave %>%  
    mutate(  
      YEAR = as.integer(YEAR),  
      MONTH = as.integer(MONTH),  
      DAY = as.integer(DAY),  
      DISTANCE = as.numeric(DISTANCE),  
      AIR_TIME = as.numeric(AIR_TIME),  
      SCHEDULED_DEPARTURE = as.numeric(SCHEDULED_DEPARTURE),  
      data_partida = make_date(YEAR, MONTH, DAY),  
      hora_partida_texto = sprintf("%04d", as.integer(SCHEDULED_DEPARTURE)), # SCHEDULED_DEPARTURE  
      hora_partida_texto = paste0(substr(hora_partida_texto, 1, 2), ":", substr(hora_partida_texto, 3, 4)),  
      data_hora_partida = ymd_hm(paste(data_partida, hora_partida_texto)),  
      velocidade_media_mph = DISTANCE / (AIR_TIME / 60)  
    ) %>%  
    arrange(data_hora_partida)  
  
  # airports.csv e pequeno, da pra ler direto  
  tabela_aeroportos = read_csv(aeroportos, col_types = cols(  
    IATA_CODE = col_character(),  
    LATITUDE = col_double(),  
    LONGITUDE = col_double()  
  )) %>%  
    select(IATA_CODE, AIRPORT, LATITUDE, LONGITUDE)  
  
  # junta lat/long de origem e destino em cada voo
```

```

tabela_trajeto = voos_aeronave %>%
  left_join(tabela_aeroportos, by = c("ORIGIN_AIRPORT" = "IATA_CODE")) %>%
  rename(lat_origem = LATITUDE, lon_origem = LONGITUDE, nome_origem = AIRPORT) %>%
  left_join(tabela_aeroportos, by = c("DESTINATION_AIRPORT" = "IATA_CODE")) %>%
  rename(lat_destino = LATITUDE, lon_destino = LONGITUDE, nome_destino = AIRPORT) %>%
  filter(!is.na(lat_origem), !is.na(lat_destino)) %>%
  arrange(data_hora_partida) %>%
  mutate(ordem_do_voo = row_number())

# cor = ordem cronologica do trecho, espessura = velocidade media do voo
paleta_cronologica = colorNumeric(palette = "viridis", domain = tabela_trajeto$ordem_do_voo)
faixa_velocidade = range(tabela_trajeto$velocidade_media_mph, na.rm = TRUE)

mapa_trajeto = leaflet(data = tabela_trajeto) %>%
  addProviderTiles(providers$CartoDB.Positron) %>%
  addCircleMarkers(
    lng = ~lon_origem, lat = ~lat_origem,
    radius = 4, color = "black", fillOpacity = 0.8,
    popup = ~paste0(nome_origem, " (", ORIGIN_AIRPORT, ")")
  )

for (i in seq_len(nrow(tabela_trajeto))) {
  trecho = tabela_trajeto[i, ]

  espessura_linha = scales::rescale(
    trecho$velocidade_media_mph,
    to = c(1, 8),
    from = faixa_velocidade
  )

  mapa_trajeto = mapa_trajeto %>%
    addPolylines(
      lng = c(trecho$lon_origem, trecho$lon_destino),
      lat = c(trecho$lat_origem, trecho$lat_destino),
      weight = espessura_linha,
      color = paleta_cronologica(trecho$ordem_do_voo),
      opacity = 0.8,
      popup = paste0(
        "Voo ", trecho$ordem_do_voo, ": ", trecho$ORIGIN_AIRPORT, " -> ", trecho$DESTINATION_AIRPORT, "  
",
        "Data: ", trecho$data_partida, "<br>",
        "Velocidade media: ", round(trecho$velocidade_media_mph, 1), " mph"
      )
    )
}

```

```

    )
  }

  mapa_trajeto = mapa_trajeto %>%
    addLegend(
      position = "bottomright",
      pal = paleta_cronologica,
      values = tabela_trajeto$ordem_do_voo,
      title = "Ordem cronologica do voo"
    )

  return(list(
    tabela = tabela_trajeto,
    mapa = mapa_trajeto
  ))
}

```

E por último, o exemplo de chamada:

```

resultado = analisa_aeronave(
  tail_number = "N431WN",
  arquivo = file.path("flights.csv.zip"),
  aeroportos = file.path("airports.csv")
)

resultado$tabela

```

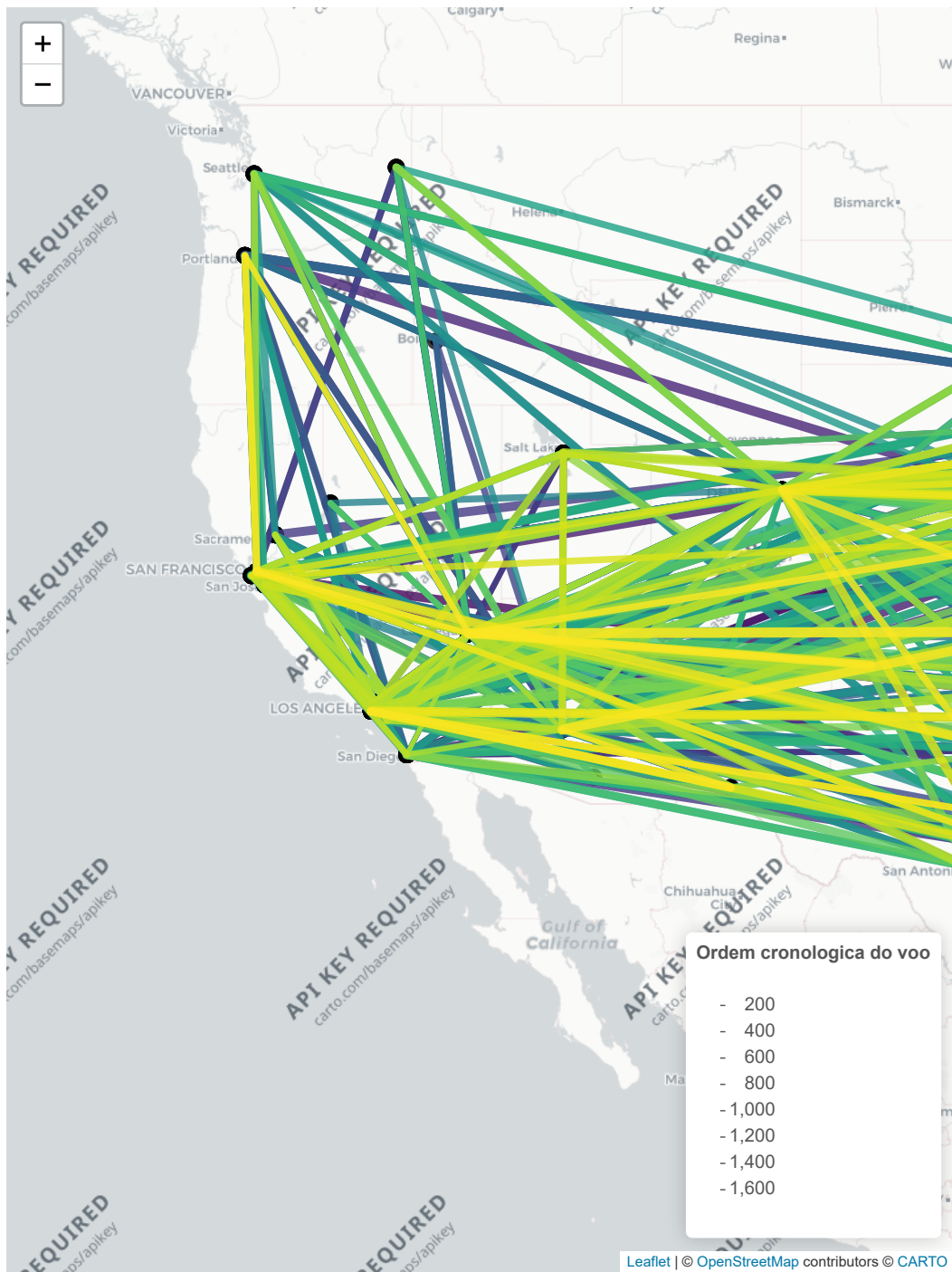
A tibble: 1,780 x 42

	YEAR	MONTH	DAY	DAY_OF_WEEK	AIRLINE	FLIGHT_NUMBER	TAIL_NUMBER
	<int>	<int>	<int>	<chr>	<chr>	<chr>	<chr>
1	2015	1	1	4	WN	1762	N431WN
2	2015	1	1	4	WN	3564	N431WN
3	2015	1	1	4	WN	3564	N431WN
4	2015	1	1	4	WN	3564	N431WN
5	2015	1	2	5	WN	1793	N431WN
6	2015	1	2	5	WN	1793	N431WN
7	2015	1	2	5	WN	1793	N431WN
8	2015	1	2	5	WN	1793	N431WN
9	2015	1	2	5	WN	1793	N431WN
10	2015	1	2	5	WN	3144	N431WN

i 1,770 more rows

```
# i 35 more variables: ORIGIN_AIRPORT <chr>, DESTINATION_AIRPORT <chr>,  
#   SCHEDULED_DEPARTURE <dbl>, DEPARTURE_TIME <chr>, DEPARTURE_DELAY <chr>,  
#   TAXI_OUT <chr>, WHEELS_OFF <chr>, SCHEDULED_TIME <chr>, ELAPSED_TIME <chr>,  
#   AIR_TIME <dbl>, DISTANCE <dbl>, WHEELS_ON <chr>, TAXI_IN <chr>,  
#   SCHEDULED_ARRIVAL <chr>, ARRIVAL_TIME <chr>, ARRIVAL_DELAY <chr>,  
#   DIVERTED <chr>, CANCELLED <chr>, CANCELLATION_REASON <chr>, ...
```

```
resultado$mapa
```



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-09-16 23:26:30 -03"
```